핵심 알고리즘 구현

개요

libiop 라이브러리의 핵심 알고리즘들은 현대적인 C++ 기법을 활용하여 효율성과 안전성을 동시에 달성합니다. 이 문서에서는 머클 트리, 다항식 연산, 병렬 처리 등 주요 알고리즘의 구현 방식과 최적화 기법을 상세히 분석합니다.

머클 트리 구현과 해시 함수 추상화

머클 트리 아키텍처

libiop의 머클 트리는 **제네릭 해시 함수 추상화**를 통해 다양한 해시 함수를 지원합니다:

```
template<typename FieldT, typename hash_digest_type>
class merkle_tree {
protected:
std::shared_ptr<leafhash<FieldT, hash_digest_type>> leaf_hasher_;
two_to_one_hash_function<hash_digest_type> node_hasher_;
std::vector<hash_digest_type> inner_nodes_;
std::vector<zk_salt_type> zk_leaf_randomness_elements_;
};
```

이 설계의 핵심 특징:

- **타입 안전성**: 컴파일 타임에 해시 타입 검증

- **성능 최적화**: 함수 포인터 대신 템플릿 특수화 사용
- **확장성**: 새로운 해시 함수 쉽게 추가 가능

해시 함수 추상화 계층

해시 함수는 다음과 같은 계층적 추상화를 통해 구현됩니다:

```
// 기본 해시체인 인터페이스
template<typename FieldT, typename MT_root_type>
class hashchain {
public:
virtual void absorb(const MT_root_type new_input) = 0;
virtual void absorb(const std::vector<FieldT> &new_input) = 0;
virtual std::vector<FieldT> squeeze(size_t num_elements) = 0;
virtual MT_root_type squeeze_root_type() = 0;
};

// 리프 해시 인터페이스
template<typename FieldT, typename leaf_hash_type>
class leafhash {
public:
virtual leaf_hash_type hash(const std::vector<FieldT> &leaf) = 0;
virtual leaf_hash_type zk_hash(const std::vector<FieldT> &leaf,
const zk_salt_type &zk_salt) = 0;
```

```
};
```

지원되는 해시 함수들

라이브러리는 다음과 같은 해시 함수들을 지원합니다:

1. **Blake2b**: 바이너리 해시 함수

- 높은 성능과 보안성
- 표준 암호학적 해시 함수

2. **Poseidon**: 대수적 해시 함수

- zkSNARK 친화적 설계
- 낮은 곱셈 복잡도

3. **Rescue**: 대수적 해시 함수

- MDS 행렬 기반 구조
- 높은 보안 마진

영지식 머클 트리

영지식성을 위한 랜덤성 추가:

```
void merkle_tree<FieldT, hash_digest_type>::sample_leaf_randomness() {
if (this->make_zk_) {
this->zk_leaf_randomness_elements_.resize(this->num_leaves_);
for (size_t i = 0; i < this->num_leaves_; ++i) {
// 보안 매개변수의 2배만큼 랜덤 바이트 생성
this->zk_leaf_randomness_elements_[i] =
sample_random_salt(2 * this->security_parameter_);
```

```
}
}
}
```

집합 멤버십 증명

효율적인 다중 쿼리 증명:

```
template<typename hash_digest_type>
struct merkle_tree_set_membership_proof {
std::vector<hash_digest_type> auxiliary_hashes;
std::vector<zk_salt_type> randomness_hashes;
std::size_t  size_in_bytes() const {
return std::accumulate(auxiliary_hashes.begin(), auxiliary_hashes.end(), 0,
[](const std::size_t  av, const hash_digest_type &h) {
return av + get_hash_size<hash_digest_type>(h);
});
}
};
```

다항식 연산 최적화와 메모리 관리

다항식 클래스 계층

다항식 연산은 다음과 같은 계층 구조로 구현됩니다:

```
template<typename FieldT>
class polynomial_base {
public:
virtual std::size_t  degree() const = 0;
virtual FieldT evaluation_at_point(const FieldT &evalpoint) const = 0;
virtual std::vector<FieldT> evaluations_over_field_subset(
const field_subset<FieldT> &S) const = 0;
};


template<typename FieldT>
class polynomial : public polynomial_base<FieldT> {
protected:
std::vector<FieldT> coefficients_;
// 메모리 효율성을 위한 최적화 메서드들
void  multiply_coefficients_by(const FieldT &other);
void  add_coefficients_of(const polynomial<FieldT> &other);
};
```

FFT 기반 다항식 연산

덧셈 FFT (Additive FFT)

Gao-Mateer 알고리즘을 사용한 덧셈 FFT:

```
template<typename FieldT>
std::vector<FieldT> additive_FFT(const std::vector<FieldT> &poly_coeffs,
const affine_subspace<FieldT> &domain) {
// Gao-Mateer 알고리즘 구현
// O(n log n) 복잡도로 덧셈 부공간에서 다항식 평가
}
```

특징:

- **이진 확장 체 최적화**: 특성 2 체에서 효율적
- **메모리 지역성**: 캐시 친화적 메모리 접근 패턴
- **병렬화 가능**: 독립적인 부분 계산들

곱셈 FFT (Multiplicative FFT)

Cooley-Tukey 알고리즘 기반:

```
template<typename FieldT>
std::vector<FieldT> multiplicative_FFT(const std::vector<FieldT> &poly_coeffs,
const multiplicative_coset<FieldT> &domain) {
// libfqfft 라이브러리 활용
// 표준 FFT 알고리즘으로 곱셈 코셋에서 평가
}
```

라그랑주 보간 최적화

캐시 기반 라그랑주 계수

```
template<typename FieldT>
class lagrange_cache {
private:
const field_subset<FieldT> domain_;
const vanishing_polynomial<FieldT> vp_;
// 캐시 최적화
const  bool cache_evaluations_;
bool called_;
FieldT last_interpolation_point_;
std::vector<FieldT> last_interpolation_result_;
public:
std::vector<FieldT> coefficients_for(const FieldT &interpolation_point);
};
```

최적화 기법:

- **결과 캐싱**: 동일한 점에 대한 반복 계산 방지
- **전처리**: 소멸 다항식 미리 계산
- **메모리 풀링**: 임시 벡터 재사용

메모리 관리 전략

RAII 기반 리소스 관리

```
template<typename FieldT>
```

```
class polynomial {
public:
void reserve(const std::size_t degree_bound) {
coefficients_.reserve(degree_bound + 1);
}
void set_degree(const std::size_t degree_bound, const bool truncate=false) {
if (degree_bound < coefficients_.size() && !truncate) {
throw std::invalid_argument("Cannot shrink without truncation");
}
coefficients_.resize(degree_bound + 1, FieldT::zero());
}
};
```

메모리 풀 활용

대용량 다항식 연산을 위한 메모리 풀:

```
class polynomial_memory_pool {
std::vector<std::vector<FieldT>> available_vectors_;
std::mutex pool_mutex_;
public:
std::vector<FieldT> acquire(size_t size) {
std::lock_guard<std::mutex> lock(pool_mutex_);
// 적절한 크기의 벡터 반환 또는 새로 생성
}
```

```
void release(std::vector<FieldT>&& vec) {
std::lock_guard<std::mutex> lock(pool_mutex_);
available_vectors_.push_back(std::move(vec));
}
};
```

병렬 처리와 SIMD 최적화 기법

병렬 FFT 구현

OpenMP 기반 병렬화

```
template<typename FieldT>
std::vector<FieldT> parallel_additive_FFT(
const std::vector<FieldT> &poly_coeffs,
const affine_subspace<FieldT> &domain) {
const size_t n = domain.num_elements();
std::vector<FieldT> result(n);
#pragma omp parallel for
for (size_t i = 0; i < n; ++i) {
// 각 평가점에서 독립적으로 계산
result[i] = evaluate_at_point(poly_coeffs, domain.element_by_index(i));
}
return result;
```

```
}
```

작업 분할 전략

```
class parallel_work_scheduler {
const size_t num_threads_;
const size_t chunk_size_;
public:
template<typename WorkFunc>
void schedule_work(size_t total_work, WorkFunc work_func) {
const size_t chunks = (total_work + chunk_size_ - 1) / chunk_size_;
#pragma omp parallel for num_threads(num_threads_)
for (size_t chunk = 0; chunk < chunks; ++chunk) {
const size_t start = chunk * chunk_size_;
const size_t end = std::min(start + chunk_size_, total_work);
work_func(start, end);
}
}
};
```

SIMD 최적화

벡터화된 필드 연산

```
template<typename FieldT>
void vectorized_field_multiply(
const std::vector<FieldT> &a,
const std::vector<FieldT> &b,
std::vector<FieldT> &result) {
const size_t n = a.size();
// SIMD 명령어 활용 (컴파일러 자동 벡터화 유도)
#pragma omp simd
for (size_t i = 0; i < n; ++i) {
result[i] = a[i] * b[i];
}
}
```

캐시 최적화된 행렬 연산

```
template<typename FieldT>
void cache_optimized_matrix_vector_multiply(
const std::vector<std::vector<FieldT>> &matrix,
const std::vector<FieldT> &vector,
std::vector<FieldT> &result) {
const size_t rows = matrix.size();
const size_t cols = vector.size();
const size_t block_size = 64; // 캐시 라인 크기에 맞춤
```

```
for (size_t i = 0; i < rows; i += block_size) {
for (size_t j = 0; j < cols; j += block_size) {
// 블록 단위로 계산하여 캐시 미스 최소화
const  size_t i_end = std::min(i + block_size, rows);
const  size_t j_end = std::min(j + block_size, cols);
for (size_t ii = i; ii < i_end; ++ii) {
for (size_t jj = j; jj < j_end; ++jj) {
result[ii] += matrix[ii][jj] * vector[jj];
}
}
}
}
}
```

비동기 처리 패턴

파이프라인 병렬성

```
template<typename FieldT>
class polynomial_evaluation_pipeline {
std::queue<std::future<std::vector<FieldT>>> pending_evaluations_;
std::thread_pool pool_;
public:
std::future<std::vector<FieldT>> submit_evaluation(
```

```
const std::vector<FieldT> &coeffs,
const field_subset<FieldT> &domain) {
return  pool_.submit([coeffs, domain]() {
return  FFT_over_field_subset(coeffs, domain);
});
}
std::vector<std::vector<FieldT>> collect_results() {
std::vector<std::vector<FieldT>> results;
while (!pending_evaluations_.empty()) {
results.push_back(pending_evaluations_.front().get());
pending_evaluations_.pop();
}
return results;
}
};
```

성능 프로파일링과 최적화

실행 시간 측정

```
class performance_profiler {
std::map<std::string, std::chrono::duration<double>> timings_;
public:
template<typename Func>
```

```
auto  time_execution(const std::string &name, Func func) {

auto start = std::chrono::high_resolution_clock::now();

auto result = func();

auto end = std::chrono::high_resolution_clock::now();

timings_[name] += (end - start);

return result;

}

void  print_profile() const {

for (const  auto &[name, duration] : timings_) {

std::cout << name << ": " << duration.count() << "s\n";

}

}

};
```

이러한 핵심 알고리즘들의 최적화된 구현은 libiop가 대규모 zkSNARK 증명을 효율적으로 생성하고 검증할 수 있는 기반을 제공합니다. 특히 병렬 처리와 메모리 최적화를 통해 실용적인 성능을 달성하고 있습니다.